

Lab Reflection — WalkMates

Lab: 3

Pair: Adam and Carl

Repo: [GitHub Repo](#)

1. What we did

We extended the `promptIncludesStructuredFields()` test in `MatchExplanationServiceTest.java`, regarding requirement **FR-5.1**, by expanding the testing of the prompt structure to make sure it includes "Listing base rate" and data delimiters for the free-text description.

To test the **FR-5.2** requirement, we created three tests: `fallsBackOnLlmTimeout()`, `fallsBackOnNullResponse()` and `fallsBackOnBlankResponse()`, in order to test the deterministic **fallback** and so that no exceptions are leaked. This was done by mocking the `LlmClient` to throw an `LlmTimeoutException`, return a null/blank response, and throw an `LlmException` respectively.

To test the **FR-5.3** requirement, we created two tests; one for each metamorphic relation. The first test was created to test irrelevant details. It was done by creating a list with candidate-listings and run it against the `recommendBestMatch()` method. One of the candidates was then modified to contain an irrelevant sentence in its description. We then ran the `recommendBestMatch()` method again and compared the new result to the original one.

The second test was created to test the chosen listing when the order of the list changes. This was done by creating a list with candidate-listings and run it against the `recommendBestMatch()` method, and then reversing the list and run it again, comparing the two outcomes.

To test prompt-injection robustness, we created a test that builds a prompt using `MatchExplanationService.buildPrompt()`, with a description containing malicious instructions. We then asserted that the description was contained within the data delimiters, and that the prompt kept its original instructions.

Finally, we tested the `GET /api/match/{seekerId}/explain` endpoint to return a 200 status code and a JSON body when a seeker and listing exist.

2. What we found

We have learnt about metamorphic testing and metamorphic relations and why they matter, especially in the context of ML-systems where failures often are not caused by errors in the code.

3. AI use (be honest — it doesn't lower your grade)

We mainly used AI to check for errors and mistakes in the tests we wrote and asked AI for advice. We also used AI to summarize the two literatures we chose to review in part B (Research trend mini-review). From the summarizations, we kept what we considered to be most relevant to what we already had done and learned in part A of the lab.

4. Judgment

We used AI to summarize the two sources we chose to review in part B. We did not read through the entire sources ourselves and decided to put our trust in the AI tool. This saved us a lot of time, but we can't be 100% sure that the summaries were completely accurate. We made this judgment based on the time we saved but at the cost of potentially missing out on important details.

5. What we'd test next

We would probably extend the metamorphic testing of the `recommendBestMatch()` method to make sure it behaves as expected in a larger variety of situations.
